

Anthony Timmel
December 12, 2025

Contents

1	Aufgabe Teil 1	3
1.1	Anforderungen	3
1.2	Hardware	3
1.2.1	Bauelemente	3
1.2.2	Vernetzung der Bauelemente	3
1.3	Programmcode	4

1 Aufgabe Teil 1

1.1 Anforderungen

Bevor das cyber-physische System (CPS) in das interne Netzwerk integriert werden kann sollen sie testen, ob die Möglichkeit besteht mit einem ESP32 als Grundlage des CPS, die Temperatur und die Luftfeuchtigkeit mit einem Sensor (DHT22) einzulesen und zu verarbeiten. Für die sekundliche Übermittlung der Daten an den später auszuwertenden MQTT-Client sollen die Sensorwerte in das Datenformat JSON abgelegt werden.

Zusatz: Greifen Sie auf den Inhalt des in MicroPython erstellten JSON-Strings zu und geben Sie die Sensordaten mit der print-Funktion im Terminal aus.

1.2 Die Vernetzung der Hardware auf dem Steckbrett

1.2.1 Bauelemente

ESP32

Der *ESP32* befindet sich bereits vorinstalliert auf dem Steckbrett und kann, durch das Hinzufügen des Stromversorgungs- & Datenübertragungskabels lässt sich die Einheit, von dem Computer aus Steuern.

Verteilung der Pin-Nutzung auf dem ESP32

Pin	Aufgabe
3V3	Stromversorgung des Bauteils
GND	Die Masse des ESP32
G4	Datenübertragungsport für die Messdaten es Bauteils

DHT22

Das Erweiterungsmodul *DHT22* ist ein Modul, was es ermöglicht, die Raumtemperatur sowie die Luftfeuchtigkeit in der Umgebung festzustellen.

Verteilung der Pin-Nutzung auf dem DHT22

Pin	Aufgabe
VCC	Stromversorgung des Bauteils
GROUND	Die Masse des DHT22
DATA	Datenübertragungsport für die Messdaten es Bauteils

1.2.2 Vernetzung der Bauelemente

Der *ESP32* ist mit dem Pin **G4** mit dem *DHT22* **DATA** Pin verbunden. Die Stromversorgung des *DHT22* geht über den **VCC** Pin des Bauteils, dieser ist mit der Stromversorgung von dem **3V3** Pin des *ESP32* verbunden. Der **GROUND** Pin des Bauteils, ist mit der Masse **GND** des *ESP32* zusammengeschlossen, damit die Schaltung elektrotechnisch Funktionstüchtig werden kann.

1.3 Programmcode für den ESP32

Funktion, damit die Daten für die Luftfeuchtigkeit & Raum-Temperatur abgegriffen werden

```
1 def get_device_temp_and_humid():
2     dht_device = DHT22(Pin(4, Pin.IN))
3     dht_device.measure()
4     temp = dht_device.temperature()
5     humid = dht_device.humidity()
6
7     return temp, humid
```

Mithilfe dieser Funktion, ist es uns möglich, die **Temperatur** und **Luftfeuchtigkeit** an dem **aktuellen Ort** zu bestimmen. Dafür sendet das Modul *DHT22* seine Daten an den Pin **G4**, diesen haben wir zuvor ([Pinverteilung ESP32](#)) definiert.

Durch **DHT22** aus der Datei **dht** erhalten wir die Möglichkeit, Daten über eine Erweiterung des Protokolls abzugreifen. Dafür haben wir den Input-Pin der DHT22 Klasse `DHT22(Pin(4, Pin.IN))` bei der Initialisierung übergeben. Im Anschluss ist es uns nun möglich, eine Messung mithilfe von `dht_device.measure()` können wir das Modul auffordern, eine Messung der Temperatur sowie Luftfeuchtigkeit vorzunehmen. Im Anschluss erhalten wir die Temperatur sowie Luftfeuchtigkeit über die jeweilige Methode des Objektes und geben diese am Ende der Funktion als **Pair** zurück.

While-Schleife, die als Hauptprozess der Anwendung läuft, und sekundlich den Prozess wiederholt

```
1 while True:
2     temp, humid = get_device_temp_and_humid()
3
4     raw_json_object = {
5         "Temperatur": temp,
6         "Luftfeuchtigkeit": humid
7     }
8
9     json_object = json.dumps(raw_json_object)
10
11     print(json_object)
12     sleep(1)
```
